

Todo List Uygulaması

1. Proje Tanımı

Uygulamanın backend tarafı ASP.NET Core Web API ile geliştirilecektir. Veritabanı işlemleri Entity Framework Core ve Code First yaklaşımıyla gerçekleştirilecektir.

Frontend tarafı Angular ile geliştirilecek; kullanıcı arayüzünde PrimeNG bileşenleri ve PrimeFlex yardımcı CSS sınıfları kullanılacaktır.

2. Kullanılacak Teknolojiler

Backend

- ASP.NET Core Web API
- Entity Framework Core
- Code First yaklaşımı
- SQL Server
- LINQ
- Dependency Injection
- Swagger / OpenAPI
- FluentValidation veya Data Annotations
- AutoMapper veya manuel DTO eşleştirme
- Global exception handling
- Repository ve Service katmanları

Frontend

- Angular
 - TypeScript
 - PrimeNG
 - PrimeFlex
 - PrimeIcons
 - Angular Reactive Forms
 - HttpClient
 - RxJS
 - Toast mesajları
 - Confirmation Dialog
 - Responsive tasarım
-

3. Proje Amaçları

Uygulama aşağıdaki temel ihtiyaçları karşılamalıdır:

1. Yeni görev oluşturma
 2. Mevcut görevi güncelleme
 3. Görevi silme
 4. Görevi tamamlandı olarak işaretleme
 5. Tamamlanan görevi tekrar aktif duruma alma
 6. Görevleri listeleme
 7. Duruma göre görev filtreleme
 8. Önceliğe göre görev filtreleme
 9. Görev başlığı içinde arama yapma
 10. Görevleri oluşturulma tarihi veya son teslim tarihine göre sıralama
 11. Mobil, tablet ve masaüstü ekranlarda responsive çalışma
-

4. Fonksiyonel Gereksinimler

4.1 Görev Oluşturma

Kullanıcı yeni bir görev oluşturabilmelidir.

Görev oluşturma alanları:

- Başlık
- Açıklama
- Öncelik
- Son teslim tarihi
- Kategori
- Tamamlanma durumu

Başlık alanı zorunlu olmalıdır.

Başlık en az 3, en fazla 150 karakter olmalıdır.

Açıklama en fazla 1000 karakter olmalıdır.

Son teslim tarihi geçmiş bir tarih olarak seçilememelidir.

4.2 Görev Listeleme

Görevler varsayılan olarak oluşturulma tarihine göre yeniden eskiye sıralanmalıdır.

Her görev kartında aşağıdaki bilgiler gösterilmelidir:

- Görev başlığı
- Açıklamanın kısa özeti
- Öncelik
- Kategori
- Son teslim tarihi
- Tamamlanma durumu
- Oluşturulma tarihi
- Düzenleme butonu
- Silme butonu
- Tamamlandı işaretleme alanı

4.3 Görev Güncelleme

Kullanıcı mevcut bir görevi düzenleyebilmelidir.

Güncellenebilecek alanlar:

- Başlık
- Açıklama
- Öncelik
- Son teslim tarihi
- Kategori
- Tamamlanma durumu

Görev güncellendiğinde UpdatedAt alanı otomatik olarak güncellenmelidir.

4.4 Görev Silme

Kullanıcı bir görevi silebilmelidir.

Silme işleminden önce PrimeNG Confirmation Dialog gösterilmelidir.

Örnek mesaj:

“Bu görevi silmek istediğinize emin misiniz?”

Kullanıcı işlemi onaylarsa kayıt silinmelidir.

İlk sürümde fiziksel silme kullanılabilir. İleri aşamada soft delete yapısına geçilebilir.

4.5 Görev Tamamlama

Kullanıcı görev kartı üzerindeki checkbox alanından görevi tamamlandı olarak işaretleyebilmelidir.

Görev tamamlandığında:

- IsCompleted alanı true olmalıdır.
- CompletedAt alanı güncel tarih ile doldurulmalıdır.
- Görev başlığı arayüzde üstü çizili gösterilebilir.
- Görev kartının görünümü tamamlanmış duruma uygun olarak değiştirilebilir.

Görev tekrar aktif duruma alınırsa:

- IsCompleted alanı false olmalıdır.
 - CompletedAt alanı null olmalıdır.
-

5. Filtreleme ve Arama

Kullanıcı görevleri aşağıdaki kriterlere göre filtreleyebilmelidir:

Durum filtresi

- Tümü
- Aktif görevler
- Tamamlanan görevler
- Geciken görevler

Öncelik filtresi

- Düşük
- Orta
- Yüksek
- Kritik

Kategori filtresi

- Kategori seçimine göre filtreleme
- Kategorisiz görevler

Arama

Arama işlemi aşağıdaki alanlarda yapılabilir:

- Görev başlığı
- Görev açıklaması

6. Backend Mimarisi

Backend projesinde katmanlı mimari kullanılmalıdır.

Önerilen proje yapısı:

ToDoList.sln

```
src/
├── ToDoList.Api
├── ToDoList.Application
├── ToDoList.Domain
└── ToDoList.Infrastructure
```

ToDoList.Api

Bu katmanda aşağıdaki yapılar bulunur:

- Controllers
- Middlewares
- Swagger ayarları
- Dependency Injection kayıtları
- Program.cs

ToDoList.Application

Bu katmanda aşağıdaki yapılar bulunur:

- DTO sınıfları
- Service arayüzleri
- Service implementasyonları
- Validation sınıfları
- Mapping işlemleri
- İş kuralları
- Response modelleri

ToDoList.Domain

Bu katmanda aşağıdaki yapılar bulunur:

- Entity sınıfları
- Enum sınıfları
- Domain kuralları
- Ortak entity sınıfları

TodoList.Infrastructure

Bu katmanda aşağıdaki yapılar bulunur:

- DbContext
- Entity Framework konfigürasyonları
- Repository sınıfları
- Migration dosyaları
- Veritabanı bağlantısı
- Seed işlemleri

7. Veritabanı Tasarımı

7.1 TodoItem Tablosu

Alan	Veri tipi	Zorunlu	Açıklama
Id	int	Evet	Primary Key
Title	nvarchar(150)	Evet	Görev başlığı
Description	nvarchar(1000)	Hayır	Görev açıklaması
Priority	int	Evet	Öncelik seviyesi
DueDate	datetime2	Hayır	Son teslim tarihi
IsCompleted	bit	Evet	Tamamlanma durumu
CategoryId	int	Hayır	Kategori foreign key
CreatedAt	datetime2	Evet	Oluşturulma tarihi
UpdatedAt	datetime2	Hayır	Güncellenme tarihi
IsDeleted	bit	Evet	Soft delete alanı

7.2 Category Tablosu

Alan	Veri tipi	Zorunlu	Açıklama
Id	int	Evet	Primary Key
Name	nvarchar(100)	Evet	Kategori adı
CreatedAt	datetime2	Evet	Oluşturulma tarihi
UpdatedAt	datetime2	Hayır	Güncellenme tarihi
IsDeleted	bit	Evet	Soft delete alanı

8. Entity Modelleri

BaseEntity

```
public abstract class BaseEntity
{
    public int Id { get; set; }

    public DateTime CreatedAt { get; set; }

    public DateTime? UpdatedAt { get; set; }

    public bool IsDeleted { get; set; }
}
```

TodoItem

```
public class TodoItem : BaseEntity
{
    public string Title { get; set; } = string.Empty;

    public string? Description { get; set; }

    public TodoPriority Priority { get; set; }

    public DateTime? DueDate { get; set; }

    public bool IsCompleted { get; set; }

    public int? CategoryId { get; set; }

    public Category? Category { get; set; }
}
```

Category

```
public class Category : BaseEntity
{
    public string Name { get; set; } = string.Empty;

    public string? Color { get; set; }

    public ICollection<TodoItem> TodoItems { get; set; }
    = new List<TodoItem>();
}
```

TodoPriority Enum

```
public enum TodoPriority
{
```

```
    Low = 1,  
    Medium = 2,  
    High = 3,  
    Critical = 4  
}
```

9. Entity Framework DbContext

```
public class TodoDbContext : DbContext  
{  
    public TodoDbContext(DbContextOptions<TodoDbContext> options)  
        : base(options)  
    {  
    }  
  
    public DbSet<TodoItem> TodoItems => Set<TodoItem>();  
  
    public DbSet<Category> Categories => Set<Category>();  
  
    protected override void OnModelCreating(ModelBuilder modelBuilder)  
    {  
        base.OnModelCreating(modelBuilder);  
  
        modelBuilder.ApplyConfigurationsFromAssembly(  
            typeof(TodoDbContext).Assembly  
        );  
    }  
}
```

10. Entity Konfigürasyonu

```
public class TodoItemConfiguration  
    : IEntityTypeConfiguration<TodoItem>  
{  
    public void Configure(EntityTypeBuilder<TodoItem> builder)  
    {  
        builder.ToTable("TodoItems");  
  
        builder.HasKey(x => x.Id);  
  
        builder.Property(x => x.Title)  
            .IsRequired()  
            .HasMaxLength(150);  
  
        builder.Property(x => x.Description)  
            .HasMaxLength(1000);  
    }  
}
```



```

        builder.Property(x => x.Priority)
            .IsRequired();

        builder.Property(x => x.IsCompleted)
            .HasDefaultValue(false);

        builder.Property(x => x.IsDeleted)
            .HasDefaultValue(false);

        builder.HasOne(x => x.Category)
            .WithMany(x => x.Items)
            .HasForeignKey(x => x.CategoryId)
            .OnDelete(DeleteBehavior.SetNull);

        builder.HasQueryFilter(x => !x.IsDeleted);
    }
}

```

11. Code First Migration İşlemleri

İlk migration oluşturulmalıdır:

```
dotnet ef migrations add InitialCreate
```

Veritabanı güncellenmelidir:

```
dotnet ef database update
```

Migration işlemi Infrastructure projesindeyse örnek komut:

```

dotnet ef migrations add InitialCreate \
    --project TodoList.Infrastructure \
    --startup-project TodoList.Api

```

Veritabanını güncelleme:

```

dotnet ef database update \
    --project TodoList.Infrastructure \
    --startup-project TodoList.Api

```

12. DTO Modelleri

Entity sınıfları doğrudan API üzerinden dönülmemelidir.

CreateTodoRequestDto

```
public class CreateTodoRequestDto
{
    public string Title { get; set; } = string.Empty;

    public string? Description { get; set; }

    public TodoPriority Priority { get; set; }

    public DateTime? DueDate { get; set; }

    public int? CategoryId { get; set; }
}
```

UpdateTodoRequestDto

```
public class UpdateTodoRequestDto
{
    public int Id { get; set; }

    public string Title { get; set; } = string.Empty;

    public string? Description { get; set; }

    public TodoPriority Priority { get; set; }

    public DateTime? DueDate { get; set; }

    public int? CategoryId { get; set; }
}
```

TodoResponseDto

```
public class TodoResponseDto
{
    public int Id { get; set; }

    public string Title { get; set; } = string.Empty;

    public string? Description { get; set; }

    public TodoPriority Priority { get; set; }

    public DateTime? DueDate { get; set; }

    public bool IsCompleted { get; set; }

    public int? CategoryId { get; set; }
}
```

```

    public string? CategoryName { get; set; }

    public string? CategoryColor { get; set; }

    public DateTime CreatedAt { get; set; }

    public DateTime? UpdatedAt { get; set; }
}

```

ChangeTodoStatusRequestDto

```

public class ChangeTodoStatusRequestDto
{
    public bool IsCompleted { get; set; }
}

```

13. API Endpointleri

Todo endpointleri

HTTP	Endpoint	Açıklama
GET	/api/todos	Görevleri listeler
GET	/api/todos/{id}	Görev detayını getirir
POST	/api/todos	Yeni görev oluşturur
PUT	/api/todos/{id}	Görevi günceller
PATCH	/api/todos/{id}/status	Görev durumunu değiştirir
DELETE	/api/todos/{id}	Görevi siler

Category endpointleri

HTTP	Endpoint	Açıklama
GET	/api/categories	Kategorileri listeler
POST	/api/categories	Yeni kategori oluşturur
PUT	/api/categories/{id}	Kategoriye günceller
DELETE	/api/categories/{id}	Kategoriye siler

14. Listeleme İstek Parametreleri

Görev listeleme endpointi aşağıdaki query parametrelerini desteklemelidir:

```

GET /api/todos
  ?search=rapor

```

```
&status=active
&priority=high
&categoryId=2
&sortBy=dueDate
&sortDirection=asc
&pageNumber=1
&pageSize=10
```

Örnek filtre DTO'su

```
public class TodoFilterRequestDto
{
    public string? Search { get; set; }

    public string? Status { get; set; }

    public TodoPriority? Priority { get; set; }

    public int? CategoryId { get; set; }

    public string SortBy { get; set; } = "createdAt";

    public string SortDirection { get; set; } = "desc";

    public int PageNumber { get; set; } = 1;

    public int PageSize { get; set; } = 10;
}
```

15. Standart API Response Yapısı

API yanıtlarının ortak bir model üzerinden dönmesi önerilir.

```
public class ApiResponse<T>
{
    public bool Success { get; set; }

    public string? Message { get; set; }

    public T? Data { get; set; }

    public IEnumerable<string>? Errors { get; set; }
}
```

Başarılı cevap örneği:

```
{
  "success": true,
```

```
"message": "Görev başarıyla oluşturuldu.",
"data": {
  "id": 1,
  "title": "Haftalık raporu hazırla"
},
"errors": null
}
```

Hatalı cevap örneği:

```
{
  "success": false,
  "message": "Doğrulama hatası oluştu.",
  "data": null,
  "errors": [
    "Başlık alanı zorunludur."
  ]
}
```

16. Backend Validasyonları

Görev oluşturma ve güncelleme işlemlerinde aşağıdaki validasyonlar uygulanmalıdır:

- Başlık boş olamaz.
- Başlık en az 3 karakter olmalıdır.
- Başlık en fazla 150 karakter olmalıdır.
- Açıklama en fazla 1000 karakter olmalıdır.
- Öncelik geçerli bir enum değeri olmalıdır.
- Son teslim tarihi geçmiş olamaz.
- Gönderilen kategori mevcut olmalıdır.
- Güncellenmek istenen görev mevcut olmalıdır.
- Silinmiş görev üzerinde işlem yapılamamalıdır.

17. Global Exception Handling

Beklenmeyen hataların controller içerisinde ayrı ayrı yönetilmesi yerine merkezi bir exception middleware kullanılmalıdır.

Yakalanabilecek hata türleri:

- ValidationException
- NotFoundException
- BusinessException

- UnauthorizedAccessException
- Genel sistem hataları

Production ortamında hata detayları kullanıcıya doğrudan gösterilmemelidir.

18. Angular Proje Yapısı

Önerilen Angular klasör yapısı:

```
src/app/
├── core/
│   ├── interceptors/
│   ├── guards/
│   ├── models/
│   └── services/
├── shared/
│   ├── components/
│   ├── directives/
│   └── pipes/
├── features/
│   └── todos/
│       ├── components/
│       │   ├── todo-list/
│       │   ├── todo-card/
│       │   ├── todo-form/
│       │   └── todo-filter/
│       ├── models/
│       ├── services/
│       └── pages/
│           └── todo-page/
└── app.component.ts
```

19. Angular Modelleri

```
export type TodoPriority =
  | 'low'
  | 'medium'
  | 'high'
  | 'critical';
```

```
export interface TodoItem {
  id: number;
```

```

    title: string;
    description?: string;
    priority: TodoPriority;
    dueDate?: string;
    isCompleted: boolean;
    categoryId?: number;
    categoryName?: string;
    categoryColor?: string;
    createdAt: string;
    updatedAt?: string;
}

```

20. Angular Servis Yapısı

```

@Injectable({
  providedIn: 'root'
})
export class TodoService {
  private readonly apiUrl = '/api/todos';

  constructor(private http: HttpClient) {}

  getTodos(params?: HttpParams): Observable<ApiResponse<TodoItem[]>> {
    return this.http.get<ApiResponse<TodoItem[]>>(
      this.apiUrl,
      { params }
    );
  }

  getTodoById(id: number): Observable<ApiResponse<TodoItem>> {
    return this.http.get<ApiResponse<TodoItem>>(
      `${this.apiUrl}/${id}`
    );
  }

  createTodo(
    request: CreateTodoRequest
  ): Observable<ApiResponse<TodoItem>> {
    return this.http.post<ApiResponse<TodoItem>>(
      this.apiUrl,
      request
    );
  }

  updateTodo(
    id: number,
    request: UpdateTodoRequest
  ): Observable<ApiResponse<TodoItem>> {

```

```

        return this.http.put<ApiResponse<TodoItem>>(
            `${this.apiUrl}/${id}`,
            request
        );
    }

    changeStatus(
        id: number,
        isCompleted: boolean
    ): Observable<ApiResponse<void>> {
        return this.http.patch<ApiResponse<void>>(
            `${this.apiUrl}/${id}/status`,
            { isCompleted }
        );
    }

    deleteTodo(id: number): Observable<ApiResponse<void>> {
        return this.http.delete<ApiResponse<void>>(
            `${this.apiUrl}/${id}`
        );
    }
}

```

21. Kullanılacak PrimeNG Bileşenleri

Todo List ekranında aşağıdaki PrimeNG bileşenleri kullanılabilir:

Bileşen	Kullanım amacı
p-card	Ana içerik ve görev kartları
p-inputText	Başlık ve arama alanları
p-textarea	Görev açıklaması
p-select veya p-dropdown	Öncelik ve kategori seçimi
p-datepicker veya p-calendar	Son teslim tarihi
p-checkbox	Tamamlandı durumu
p-button	Ekleme, güncelleme ve silme işlemleri
p-dialog	Görev ekleme ve düzenleme formu
p-confirmDialog	Silme onayı
p-toast	Başarı ve hata mesajları
p-tag	Öncelik ve durum gösterimi
p-progressBar	Tamamlanma oranı
p-paginator	Sayfalama
p-skeleton	Yükleme durumu

Bileşen	Kullanım amacı
p-tooltip	Buton açıklamaları
p-menu	Görev işlem menüsü

22. PrimeFlex Kullanımı

PrimeFlex aşağıdaki ihtiyaçlarda kullanılacaktır:

- Grid yerleşimi
 - Responsive kolonlar
 - Flex hizalama
 - Margin ve padding
 - Metin hizalama
 - Genişlik ve yükseklik
 - Responsive görünülük
 - Gap kullanımı
-

23. Todo List Ekran Tasarımı

Ana Todo ekranı aşağıdaki bölümlerden oluşmalıdır:

Üst bölüm

- Sayfa başlığı
- Toplam görev sayısı
- Tamamlanan görev sayısı
- Aktif görev sayısı
- Genel ilerleme çubuğu
- Yeni görev ekle butonu

Filtre bölümü

- Arama alanı
- Durum seçimi
- Öncelik seçimi
- Kategori seçimi
- Sıralama seçimi
- Filtreleri temizle butonu

Görev listesi

Her görev kartında:

- Checkbox
- Başlık
- Açıklama
- Öncelik etiketi
- Kategori etiketi
- Son teslim tarihi
- Düzenleme butonu
- Silme butonu
- İşlem menüsü

Boş liste durumu

Görev bulunamadığında kullanıcıya uygun bir empty state gösterilmelidir.

Örnek metin:

Henüz bir görev bulunmuyor. İlk görevinizi oluşturarak başlayabilirsiniz.

24. Öncelik Gösterimleri

Öncelikler PrimeNG Tag bileşeni ile gösterilebilir.

Öncelik	Severity
Düşük	success
Orta	info
Yüksek	warn
Kritik	danger

Örnek:

```
<p-tag
  [value]="getPriorityLabel(todo.priority)"
  [severity]="getPrioritySeverity(todo.priority)">
</p-tag>
```

25. Görev Ekleme ve Düzenleme Dialog'u

Görev ekleme ve düzenleme işlemleri aynı form componenti üzerinden yapılabilir.

Dialog başlığı işlem tipine göre değiştirilmelidir:

- Yeni Görev Oluştur
- Görevi Düzenle

Form alanları:

- Görev başlığı
- Açıklama
- Öncelik
- Kategori
- Son teslim tarihi

Dialog altındaki butonlar:

- İptal
- Kaydet
- Güncelle

Form gönderilirken buton loading durumuna alınmalıdır.

26. Reactive Form Yapısı

```
this.todoForm = this.formBuilder.group({
  title: [
    '',
    [
      Validators.required,
      Validators.minLength(3),
      Validators.maxLength(150)
    ]
  ],
  description: [
    '',
    [
      Validators.maxLength(1000)
    ]
  ],
  priority: [
    'medium',
    Validators.required
  ],
  dueDate: [
    null
  ],
  categoryId: [
    null
  ]
});
```

27. Kullanıcı Bildirimleri

Başarılı işlemlerde PrimeNG Toast kullanılmalıdır.

Örnek mesajlar:

- Görev başarıyla oluşturuldu.
- Görev başarıyla güncellendi.
- Görev tamamlandı.
- Görev tekrar aktif duruma alındı.
- Görev başarıyla silindi.
- Kategori başarıyla oluşturuldu.

Hatalı işlemlerde:

- İşlem sırasında bir hata oluştu.
- Görev bulunamadı.
- Form alanlarını kontrol ediniz.
- Sunucuya ulaşamadı.

29. Responsive Tasarım

Uygulama aşağıdaki ekranlarda sorunsuz çalışmalıdır:

Mobil

- Filtre alanları alt alta gösterilmelidir.
- Görev kartındaki işlemler alt satıra geçebilmelidir.
- Dialog genişliği ekran genişliğine göre ayarlanmalıdır.
- Butonlar gerektiğinde tam genişlikte gösterilmelidir.

Tablet

- Filtre alanları iki kolon hâlinde gösterilebilir.
- Görev kartları tek kolon devam edebilir.

Masaüstü

- Filtre alanları aynı satırda gösterilebilir.
- Görev detayları daha geniş gösterilebilir.
- Ana içerik maksimum genişlik ile ortalanmalıdır.

Büyük ekranlar

- İçerik kontrolsüz büyütülmemelidir.
- Ana container için maksimum genişlik kullanılmalıdır.

- Metin ve buton boyutları gereksiz şekilde büyütülmemelidir.
-

30. HTTP Interceptor

Angular tarafında merkezi hata yönetimi için interceptor kullanılmalıdır.

Interceptor aşağıdaki görevleri üstlenebilir:

- API hata cevaplarını yakalama
 - Yetkilendirme token'ı ekleme
 - 401 ve 403 durumlarını yönetme
 - Genel hata mesajı gösterme
 - Loading durumunu merkezi olarak yönetme
-
-

31. Geliştirme Aşamaları

Aşama 1 — Backend altyapısı

- Solution oluşturulması
- Katmanların oluşturulması
- Entity sınıflarının hazırlanması
- DbContext oluşturulması
- Entity konfigürasyonlarının hazırlanması
- Migration oluşturulması
- Veritabanının oluşturulması

Aşama 2 — Backend servisleri

- Repository yapısı
- Todo service
- Category service
- DTO modelleri
- Validation işlemleri
- Controller endpointleri
- Global exception handling
- Swagger testleri

Aşama 3 — Angular altyapısı

- Angular projesinin oluşturulması

- PrimeNG kurulumu
- PrimeFlex kurulumu
- PrimeIcons kurulumu
- Core ve shared yapıların hazırlanması
- API servislerinin oluşturulması
- Model sınıflarının hazırlanması

Aşama 4 — Todo ekranı

- Ana Todo List ekranı
- Filtre alanları
- Görev kartları
- Görev ekleme dialog'u
- Görev düzenleme dialog'u
- Silme confirmation dialog'u
- Toast mesajları

32. Sonuç

Bu doküman kapsamında geliştirilecek Todo List uygulaması; ASP.NET Core Web API, Entity Framework Core Code First, Angular, PrimeNG ve PrimeFlex teknolojileri kullanılarak hazırlanacaktır.

Backend tarafında katmanlı, sürdürülebilir ve test edilebilir bir mimari uygulanacaktır. Frontend tarafında responsive, kullanıcı dostu ve yeniden kullanılabilir component yapısı oluşturulacaktır.

İlk sürüm temel görev yönetimi, kategori yönetimi, filtreleme, arama ve durum değiştirme işlemlerini kapsayacaktır. İleri aşamalarda authentication, bildirim, alt görev ve takvim görünümü gibi özelliklerle genişletilebilecektir.